

Лабораторная работа №1

Дисциплина: Сетевые технологии

БАХИ СИДИ АЛИ ТЕМАССИНИ

2026-03-05

Содержание I

1. Информация
2. Выполнение лабораторной работы
3. Построение графиков в Octave
4. Разложение импульсного сигнала в частичный ряд Фурье
5. Определение спектра и параметров сигнала
6. Амплитудная модуляция
7. Кодирование сигнала. Исследование свойства самосинхронизации сигнала

Раздел 1

1. Информация

1.1 Докладчик

- БАХИ СИДИ АЛИ ТЕМАССИНИ

1.1 Докладчик

- БАХИ СИДИ АЛИ ТЕМАССИНИ
- Студент 3 курса

1.1 Докладчик

- БАХИ СИДИ АЛИ ТЕМАССИНИ
- Студент 3 курса
- Факультет: Фундаментальная информатика и информационные технологии

1.1 Докладчик

- БАХИ СИДИ АЛИ ТЕМАССИНИ
- Студент 3 курса
- Факультет: Фундаментальная информатика и информационные технологии
- Российский университет дружбы народов

1.1 Докладчик

- БАХИ СИДИ АЛИ ТЕМАССИНИ
- Студент 3 курса
- Факультет: Фундаментальная информатика и информационные технологии
- Российский университет дружбы народов
- Email: 1032234211@pfur.ru

1.1 Докладчик

- БАХИ СИДИ АЛИ ТЕМАССИНИ
- Студент 3 курса
- Факультет: Фундаментальная информатика и информационные технологии
- Российский университет дружбы народов
- Email: 1032234211@pfur.ru
- GitHub: https://github.com/sidiali2030/study_2025-2026_nettech

1.2 Цели и задачи

- Изучение методов кодирования и модуляции сигналов в Octave

1.2 Цели и задачи

- Изучение методов кодирования и модуляции сигналов в Octave
- Определение спектра и параметров сигнала

1.2 Цели и задачи

- Изучение методов кодирования и модуляции сигналов в Octave
- Определение спектра и параметров сигнала
- Демонстрация принципов амплитудной модуляции

1.2 Цели и задачи

- Изучение методов кодирования и модуляции сигналов в Octave
- Определение спектра и параметров сигнала
- Демонстрация принципов амплитудной модуляции
- Исследование свойства самосинхронизации сигнала

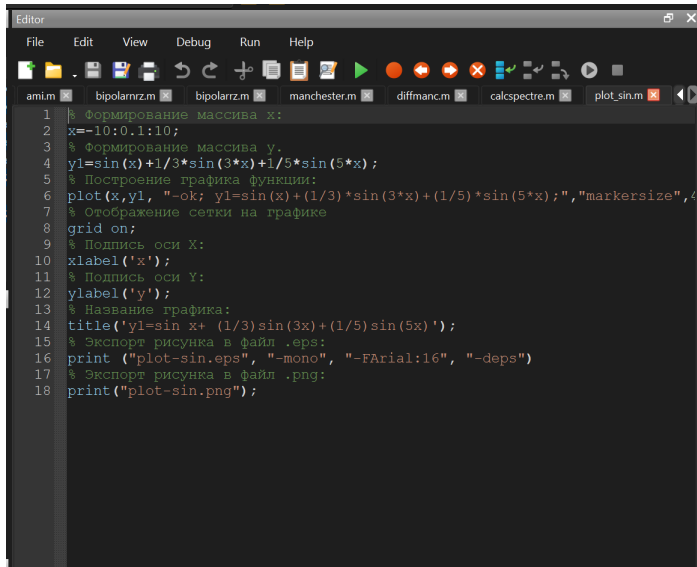
Раздел 2

2. Выполнение лабораторной работы

Раздел 3

3. Построение графиков в Octave

3.1 График y_1



```
1 % Формирование массива x:
2 x=-10:0.1:10;
3 % Формирование массива y.
4 y1=sin(x)+1/3*sin(3*x)+1/5*sin(5*x);
5 % Построение графика функции:
6 plot(x,y1, "-ok; y1=sin(x)+(1/3)*sin(3*x)+(1/5)*sin(5*x);", "markersize", 4);
7 % Отображение сетки на графике
8 grid on;
9 % Подпись оси X:
10 xlabel('x');
11 % Подпись оси Y:
12 ylabel('y');
13 % Название графика:
14 title('y1=sin x+ (1/3)sin(3x)+(1/5)sin(5x)');
15 % Экспорт рисунка в файл .eps:
16 print ("plot-sin.eps", "-mono", "-FArial:16", "-deps")
17 % Экспорт рисунка в файл .png:
18 print("plot-sin.png");
```

3.2 График у1

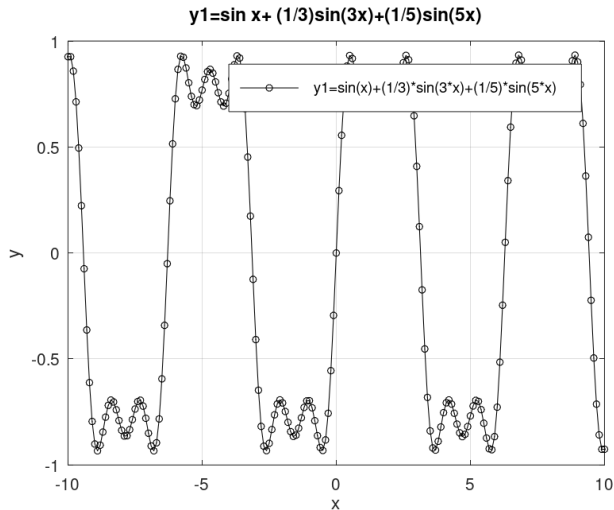


Рисунок 2: График функций y_1 на интервале $-10; 10$

3.3 Проверка файлов

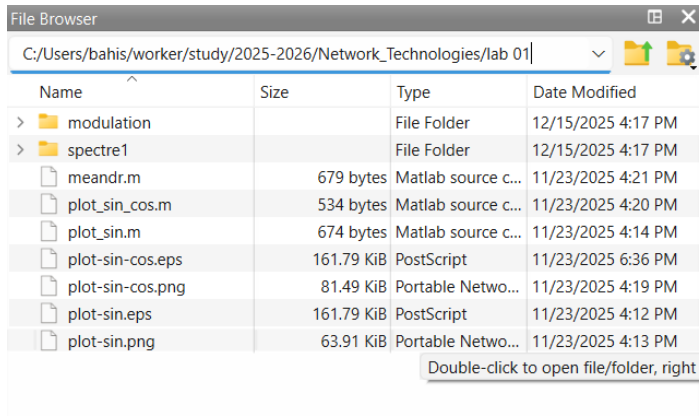
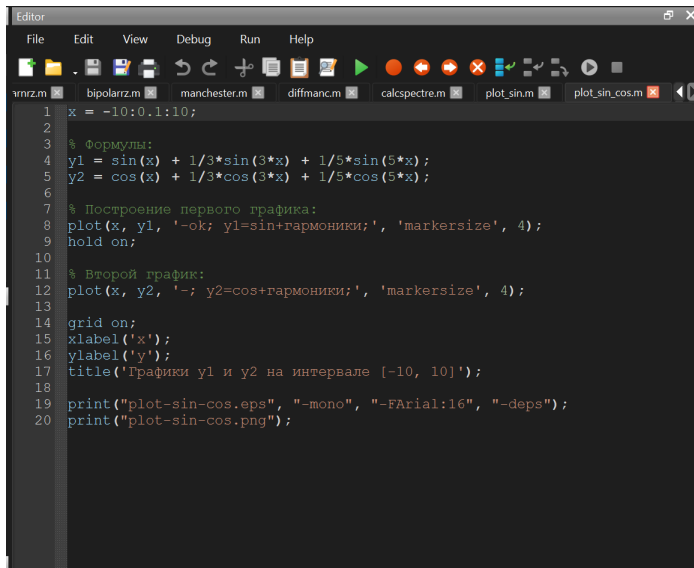


Рисунок 3: Файлы .eps, .png

3.4 Графики y_1 и y_2



```
1 x = -10:0.1:10;
2
3 % Формулы:
4 y1 = sin(x) + 1/3*sin(3*x) + 1/5*sin(5*x);
5 y2 = cos(x) + 1/3*cos(3*x) + 1/5*cos(5*x);
6
7 % Построение первого графика:
8 plot(x, y1, '-ok; y1=sin+гармоники;', 'markersize', 4);
9 hold on;
10
11 % Второй график:
12 plot(x, y2, '-; y2=cos+гармоники;', 'markersize', 4);
13
14 grid on;
15 xlabel('x');
16 ylabel('y');
17 title('Графики y1 и y2 на интервале [-10, 10]');
18
19 print("plot-sin-cos.eps", "-mono", "-FArial:16", "-deps");
20 print("plot-sin-cos.png");
```

3.5 Графики y_1 и y_2

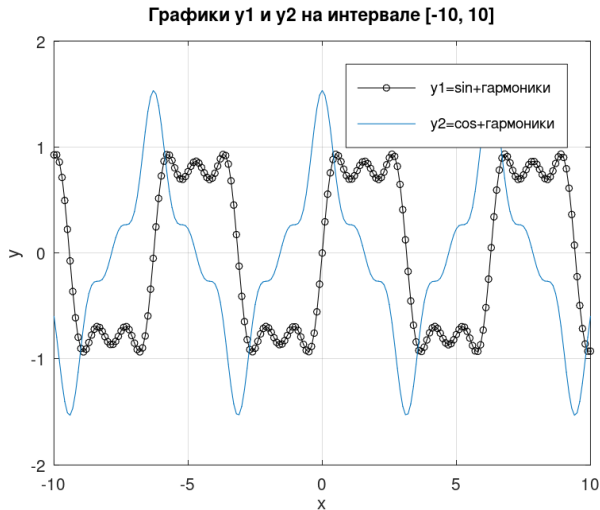


Рисунок 5: График функций y_1 и y_2 на интервале $-10; 10$

Раздел 4

4. Разложение импульсного сигнала в частичный ряд Фурье

4.1 Меандр через косинус

- Создадим новый сценарий meandr.m. В коде зададим начальные значения. Вычислим амплитуду гармоник и заполним массивы гармоник и элементов ряда. Далее задаём массив значений гармоник массив элементов ряда. Также экспортируем полученный график в файл в формате .png

```
1 % meandr.m
2 % количество отсчетов (гармоники):
3 N=8;
4 % частота дискретизации:
5 ts=1/0.01:1;
6 % значение амплитуды:
7 A=1;
8 % период:
9 T=1;
10 % амплитуда гармоник
11 nh=(1:N)*2-1;
12 % массив коэффициентов для ряда, заданного через cos:
13 Ah=2/pi ./ nh;
14 Ah(2:2:end) = -Ah(2:2:end);
15 % массив гармоник:
16 harmonics=cos(2 * pi * nh' * t/T);
17 % массив элементов ряда:
18 s1=harmonics.*repmat(Ah',1,length(t));
19
20 % суммирование ряда:
21 s2=sum(s1);
22 % построение графиков:
23 for k=1:N
24 subplot(4,2,k)
25 plot(t, s2(k,:))
26 end
27
28 print('plot-meandr-cos.png');
```

Рисунок 6: Листинг файла meandr.m

4.2 Результат

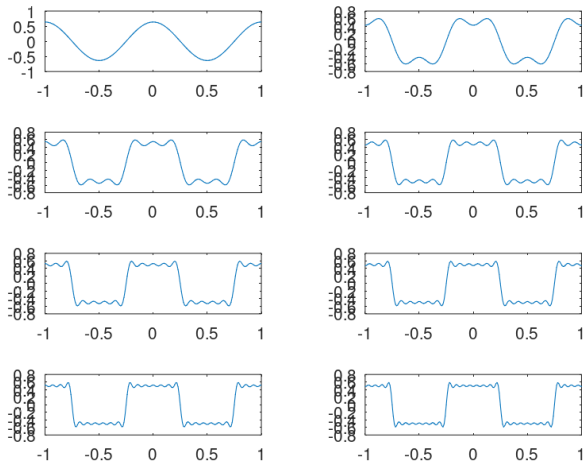


Рисунок 7: Меандр через косинусы

4.3 Меандр через синус

```
1 % meandr.m
2 % количество отсчетов (гармоник):
3 N=8;
4 % частота дискретизации:
5 t=-1:0.01:1;
6 % значение амплитуды:
7 A=1;
8 % период:
9 T=1;
10 % амплитуда гармоник
11 nh=(1:N)*2-1;
12 % массив коэффициентов для ряда, заданного через cos:
13 Am=2/pi ./ nh;
14 Am(2:2:end) = -Am(2:2:end);
15 % массив гармоник:
16 harmonics=sin(2 * pi * nh * t/T);
17 % массив элементов ряда:
18 s1=harmonics.*repmat(Am',1,length(t));
19
20 % Суммирование ряда:
21 s2=cumsum(s1);
22 % Построение графиков:
23 for k=1:N
24     subplot(4,2,k)
25     plot(t, s2(k,:))
26 end
27
28 print("plot-meandr-cos.png");
```

Рисунок 8: Листинг файла meandr.m

4.4 Результат

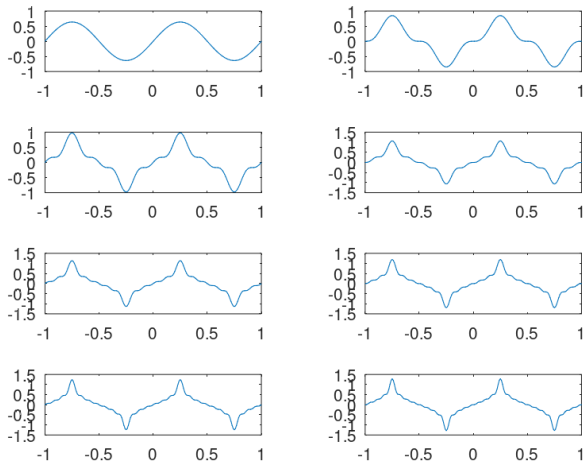


Рисунок 9: Меандр через синусы

Раздел 5

5. Определение спектра и параметров сигнала

5.1 Сигналы разной частоты

```
1 | spectrel/spectre.m
2 % Создание каталогов signal и spectre для размещения графиков:
3 mkdir 'signal';
4 mkdir 'spectre';
5 % Длина сигнала (с):
6 tmax = 0.5;
7 % Частота дискретизации (Гц) (количество отсчётов):
8 fd = 512;
9 % Частота первого сигнала (Гц):
10 f1 = 10;
11 % Частота второго сигнала (Гц):
12 f2 = 40;
13 % Амплитуда первого сигнала:
14 a1 = 1;
15 % Амплитуда второго сигнала:
16 a2 = 0.7;
17 % Массив отсчётов времени:
18 t = 0:1./fd:tmax;
19 % Спектр сигнала:
20 fd2 = fd/2;
21 % Два сигнала разной частоты:
22 signal1 = a1*sin(2*pi*t*f1);
23 signal2 = a2*sin(2*pi*t*f2);
24
25 % График 1-го сигнала:
26 plot(signal1,'b');
27 % График 2-го сигнала:
28 hold on
29 plot(signal2,'r');
30 hold off
31 title('Signal');
32 % Экспорт графика в файл в каталоге signal:
33 print 'signal/spectre.png';
34
```

Рисунок 10: Листинг файла spectre.m

5.2 Результат

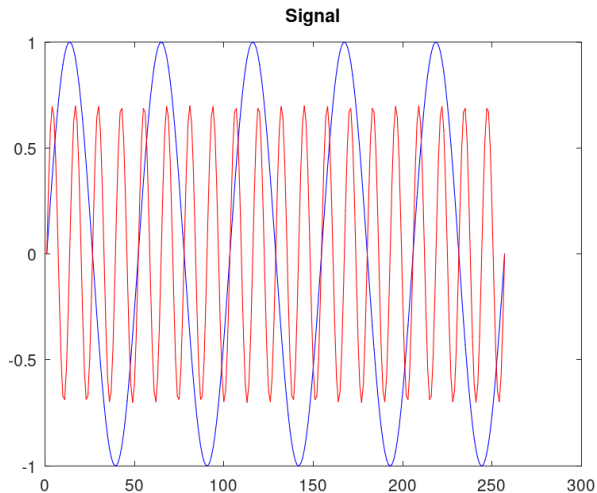


Рисунок 11: Графики сигналов разной частоты

5.3 Спектры сигналов

```
35 % Посчитаем спектр
36 % Амплитуды преобразования Фурье сигнала 1:
37 spectre1 = abs(fft(signal1,fd));
38 % Амплитуды преобразования Фурье сигнала 2:
39 spectre2 = abs(fft(signal2,fd));
40 % Построение графиков спектров сигналов:
41 plot(spectre1,'b');
42 hold on
43 plot(spectre2,'r');
44 hold off
45 title('Spectre');
46 print 'spectre/spectre.png';
47
48 % Исправление графика спектра
49 % Сетка частот:
50 f = 1000*(0:fd2)./(2*fd);
51 % Нормировка спектров по амплитуде:
52 spectre1 = 2*spectre1/fd2;
53 spectre2 = 2*spectre2/fd2;
54 % Построение графиков спектров сигналов:
55 plot(f,spectre1(1:fd2+1),'b');
56 hold on
57
58 plot(f,spectre2(1:fd2+1),'r');
59 hold off
60 xlim([0 100]);
61 title('Fixed spectre');
62 xlabel('Frequency (Hz)');
63 print 'spectre/spectre_fix.png';
64
65
```

Рисунок 12: Листинг файла spectre.m

5.4 График спектра

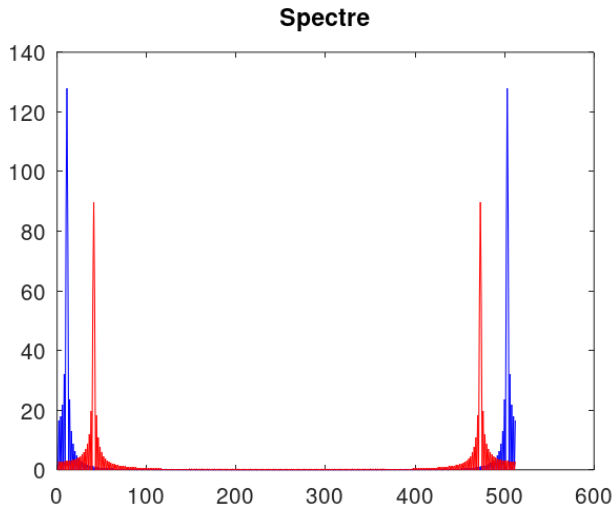


Рисунок 13: График спектра синусоидальных сигналов

5.5 Скорректированный график

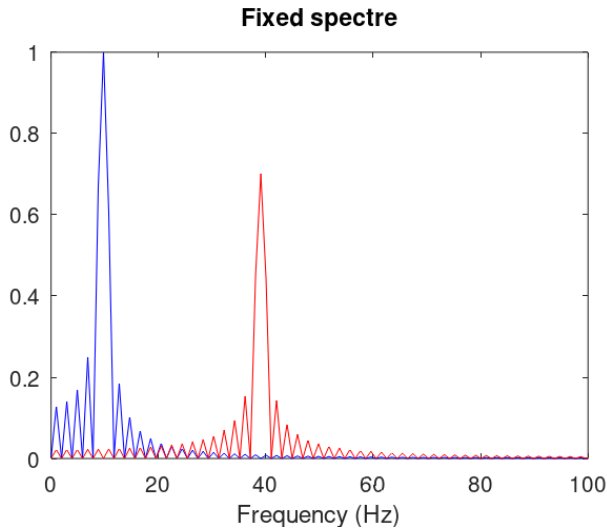


Рисунок 14: Исправленный график спектров синусоидальных сигналов

5.6 Спектр суммы

```
spectre_sum.m
1 % spectr_sum/spectre_sum.m
2 % Создание каталогов signal и спектре для размещения графиков:
3 mkdir 'signal';
4 mkdir 'spectre';
5 % Длина сигнала (с):
6 tmax = 0.5;
7 % Частота дискретизации (Гц) (количество отсчетов):
8 fd = 512;
9 % Частота первого сигнала (Гц):
10 f1 = 10;
11 % Частота второго сигнала (Гц):
12 f2 = 40;
13 % Амплитуда первого сигнала:
14 a1 = 1;
15 % Амплитуда второго сигнала:
16 a2 = 0.7;
17 % Спектр сигнала
18 fd2 = fd/2;
19 % Сумма двух сигналов (синусоиды) разной частоты:
20 % Массив отсчетов времени:
21 t = 0:1./fd:tmax;
22 signal1 = a1*sin(2*pi*t*f1);
23 signal2 = a2*sin(2*pi*t*f2);
24 signal = signal1 + signal2;
25 plot(signal);
26 title('Signal');
27 print 'signal/spectre_sum.png';
28 % Подсчет спектра:
29 % Амплитуды преобразования Фурье сигнала:
30 spectre = fft(signal,fd);
31 % Сетка частот
32 f = 1000*(0:fd2)/(2*fd);
33 % Нормировка спектра по амплитуде:
34 spectre = 2*sgt(spectre.*conj(spectre))./fd2;
35 % Построение графика спектра сигнала:
36 plot(f,spectre(1:fd2+1));
37 xlim([0 100]);
38 title('Spectre');
39 xlabel('Frequency (Hz)');
40 print 'spectre/spectre_sum.png';
41
```

Рисунок 15: Листинг файла spectre_sum.m

5.7 Сумма

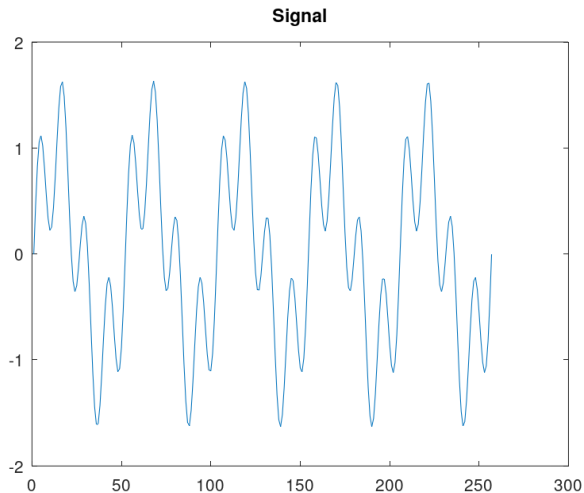


Рисунок 16: Суммарный сигнал

5.8 Спектр суммы

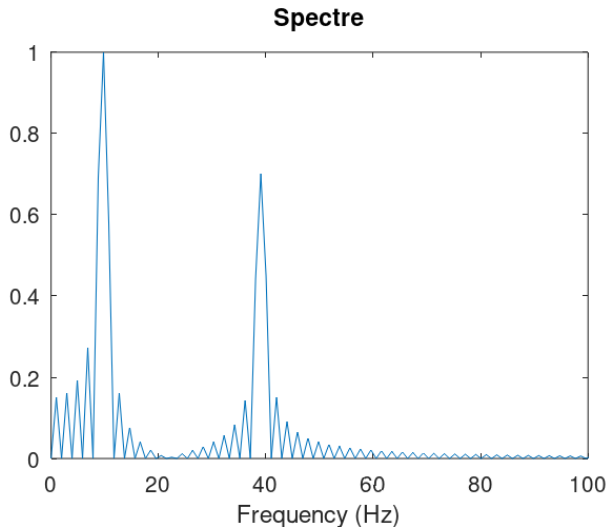


Рисунок 17: Спектр суммарного сигнала

Раздел 6

6. Амплитудная модуляция

6.1 Амплитудная модуляция

```
4 mkdir 'spectre';
5 % Модуляция синусоид с частотами 50 и 5
6 % Длина сигнала (с)
7 tmax = 0.5;
8 % Частота дискретизации (Гц) (количество отсчетов)
9 fd = 512;
10 % Частота сигнала (Гц)
11 f1 = 5;
12 % Частота несущей (Гц)
13 f2 = 50;
14 % Сигнал сигнала
15 fd2 = fd/2;
16 % Построение графиков двух сигналов (синусоиды)
17 % разной частоты
18 % Массив отсчетов времени:
19 t = 0:1./fd:tmax;
20 signal1 = sin(2*pi*t*f1);
21 signal2 = sin(2*pi*t*f2);
22 signal = signal1 .* signal2;
23 plot(signal, 'b');
24 hold on
25 % Построение отгибающей:
26 plot(signal1, 'r');
27 plot(-signal1, 'r');
28 hold off
29 title('Signal');
30 print 'signal/am.png';
31 % Расчет спектра:
32 % Амплитуды преобразования Фурье-сигнала:
33 spectre = fft(signal,fd);
34 % Сетка частот:
35 f = 1000*(0:fd2)./(2*fd);
36 % Нормировка спектра по амплитуде:
37 spectre = 2*sqrt(spectre.*conj(spectre))./fd2;
38 % Построение спектра:
39 plot(f,spectre(1:fd2+1), 'b')
40 xlim([0 100]);
41
42 title('Spectre');
43 xlabel('Frequency (Hz)');
44 print 'spectre/am.png';
```

Рисунок 18: Листинг файла am.m

6.2 Результат

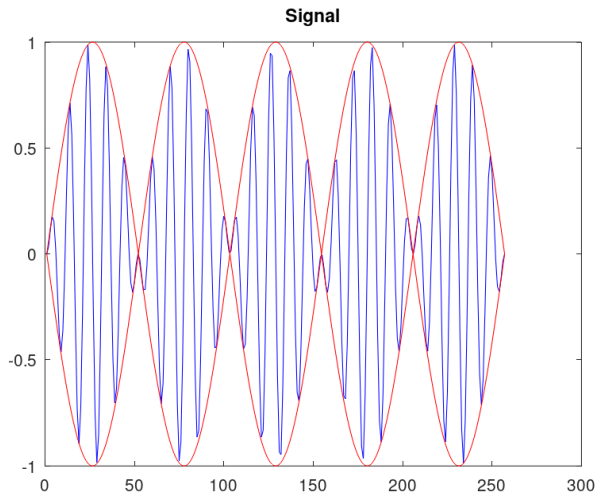


Рисунок 19: Сигнал и огибающая при амплитудной модуляции

6.3 Результат

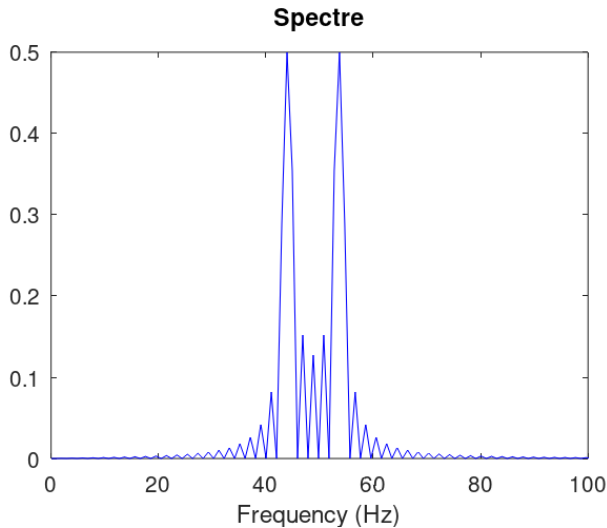


Рисунок 20: Спектр сигнала при амплитудной модуляции

Раздел 7

7. Кодирование сигнала. Исследование свойства самосинхронизации сигнала

7.1 Подготовка

- В рабочем каталоге создадим каталог coding и в нём файлы main.m, maptowave.m, unipolar.m, ami.m, bipolarnrz.m, bipolarrrz.m, manchester.m, diffmanc.m, calcspectre.m.

```
>> pkg list
Package Name | Version | Installation directory
-----|-----|-----
audio        | 2.0.9   | ...\\Octave-10.3.0\\mingw64\\share\\octave\\packages\\audio-2.0.9
biosig       | 3.9.0   | ...\\Octave-10.3.0\\mingw64\\share\\octave\\packages\\biosig-3.9.0
cfitsio      | 0.0.7   | ...\\Octave-10.3.0\\mingw64\\share\\octave\\packages\\cfitsio-0.0.7
coder        | 1.10.1  | ...\\Octave-10.3.0\\mingw64\\share\\octave\\packages\\coder-1.10.1
communications | 1.2.7   | ...\\mingw64\\share\\octave\\packages\\communications-1.2.7
control      | 4.1.3   | ...\\Octave-10.3.0\\mingw64\\share\\octave\\packages\\control-4.1.3
data-smoothing | 1.3.0   | ...\\mingw64\\share\\octave\\packages\\data-smoothing-1.3.0
database     | 2.4.4   | ...\\Octave-10.3.0\\mingw64\\share\\octave\\packages\\database-2.4.4
dataframe    | 1.2.0   | ...\\mingw64\\share\\octave\\packages\\dataframe-1.2.0
dicom        | 0.6.1   | ...\\Octave-10.3.0\\mingw64\\share\\octave\\packages\\dicom-0.6.1
financial     | 0.5.4   | ...\\mingw64\\share\\octave\\packages\\financial-0.5.4
fl-core      | 1.0.2   | ...\\Octave-10.3.0\\mingw64\\share\\octave\\packages\\fl-core-1.0.2
fuzzy-logic-toolkit | 0.6.2   | ...\\mingw64\\share\\octave\\packages\\fuzzy-logic-toolkit-0.6.2
ga           | 0.10.4  | ...\\Octave-10.3.0\\mingw64\\share\\octave\\packages\\ga-0.10.4
general      | 2.1.3   | ...\\Octave-10.3.0\\mingw64\\share\\octave\\packages\\general-2.1.3
generate_html | 0.3.3   | ...\\mingw64\\share\\octave\\packages\\generate_html-0.3.3
geometry     | 4.1.0   | ...\\Octave-10.3.0\\mingw64\\share\\octave\\packages\\geometry-4.1.0
gsl          | 2.1.1   | ...\\Octave-10.3.0\\mingw64\\share\\octave\\packages\\gsl-2.1.1
image        | 2.18.1  | ...\\Octave-10.3.0\\mingw64\\share\\octave\\packages\\image-2.18.1
image-acquisition | 0.3.3   | ...\\mingw64\\share\\octave\\packages\\image-acquisition-0.3.3
instrument-control | 0.9.5   | ...\\mingw64\\share\\octave\\packages\\instrument-control-0.9.5
interval     | 3.2.1   | ...\\Octave-10.3.0\\mingw64\\share\\octave\\packages\\interval-3.2.1
io           | 2.7.0   | ...\\Octave-10.3.0\\mingw64\\share\\octave\\packages\\io-2.7.0
linear-algebra | 2.2.3   | ...\\mingw64\\share\\octave\\packages\\linear-algebra-2.2.3
lssa        | 0.1.4   | ...\\Octave-10.3.0\\mingw64\\share\\octave\\packages\\lssa-0.1.4
ltfat        | 2.6.0   | ...\\Octave-10.3.0\\mingw64\\share\\octave\\packages\\ltfat-2.6.0
mapping      | 1.4.3   | ...\\Octave-10.3.0\\mingw64\\share\\octave\\packages\\mapping-1.4.3
matgeom     | 1.2.4   | ...\\Octave-10.3.0\\mingw64\\share\\octave\\packages\\matgeom-1.2.4
miscellaneous | 1.3.1   | ...\\mingw64\\share\\octave\\packages\\miscellaneous-1.3.1
```

7.1 Подготовка

- В рабочем каталоге создадим каталог coding и в нём файлы main.m, maptowave.m, unipolar.m, ami.m, bipolarnrz.m, bipolarrrz.m, manchester.m, diffman.m, calcspectre.m.
- В окне интерпретатора команд проверяем, установлен ли пакет расширений signal: pkg list. Так как он не установлен, то устанавливаем его: pkg list -forge и pkg install control signal

```
>> pkg list
Package Name | Version | Installation directory
-----|-----|-----
audio | 2.0.9 | ...\\Octave-10.3.0\\mingw64\\share\\octave\\packages\\audio-2.0.9
biosig | 3.9.0 | ...\\Octave-10.3.0\\mingw64\\share\\octave\\packages\\biosig-3.9.0
cfitsio | 0.0.7 | ...\\Octave-10.3.0\\mingw64\\share\\octave\\packages\\cfitsio-0.0.7
coder | 1.10.1 | ...\\Octave-10.3.0\\mingw64\\share\\octave\\packages\\coder-1.10.1
communications | 1.2.7 | ...\\mingw64\\share\\octave\\packages\\communications-1.2.7
control | 4.1.3 | ...\\Octave-10.3.0\\mingw64\\share\\octave\\packages\\control-4.1.3
data-smoothing | 1.3.0 | ...\\mingw64\\share\\octave\\packages\\data-smoothing-1.3.0
database | 2.4.4 | ...\\Octave-10.3.0\\mingw64\\share\\octave\\packages\\database-2.4.4
dataframe | 1.2.0 | ...\\mingw64\\share\\octave\\packages\\dataframe-1.2.0
dicom | 0.6.1 | ...\\Octave-10.3.0\\mingw64\\share\\octave\\packages\\dicom-0.6.1
financial | 0.5.4 | ...\\mingw64\\share\\octave\\packages\\financial-0.5.4
fl-core | 1.0.2 | ...\\Octave-10.3.0\\mingw64\\share\\octave\\packages\\fl-core-1.0.2
fuzzy-logic-toolkit | 0.6.2 | ...\\mingw64\\share\\octave\\packages\\fuzzy-logic-toolkit-0.6.2
ga | 0.10.4 | ...\\Octave-10.3.0\\mingw64\\share\\octave\\packages\\ga-0.10.4
general | 2.1.3 | ...\\Octave-10.3.0\\mingw64\\share\\octave\\packages\\general-2.1.3
generate_html | 0.3.3 | ...\\mingw64\\share\\octave\\packages\\generate_html-0.3.3
geometry | 4.1.0 | ...\\Octave-10.3.0\\mingw64\\share\\octave\\packages\\geometry-4.1.0
gsl | 2.1.1 | ...\\Octave-10.3.0\\mingw64\\share\\octave\\packages\\gsl-2.1.1
image | 2.18.1 | ...\\Octave-10.3.0\\mingw64\\share\\octave\\packages\\image-2.18.1
image-acquisition | 0.3.3 | ...\\mingw64\\share\\octave\\packages\\image-acquisition-0.3.3
instrument-control | 0.9.5 | ...\\mingw64\\share\\octave\\packages\\instrument-control-0.9.5
interval | 3.2.1 | ...\\Octave-10.3.0\\mingw64\\share\\octave\\packages\\interval-3.2.1
io | 2.7.0 | ...\\Octave-10.3.0\\mingw64\\share\\octave\\packages\\io-2.7.0
linear-algebra | 2.2.3 | ...\\mingw64\\share\\octave\\packages\\linear-algebra-2.2.3
lssa | 0.1.4 | ...\\Octave-10.3.0\\mingw64\\share\\octave\\packages\\lssa-0.1.4
ltfat | 2.6.0 | ...\\Octave-10.3.0\\mingw64\\share\\octave\\packages\\ltfat-2.6.0
mapping | 1.4.3 | ...\\Octave-10.3.0\\mingw64\\share\\octave\\packages\\mapping-1.4.3
matgeom | 1.2.4 | ...\\Octave-10.3.0\\mingw64\\share\\octave\\packages\\matgeom-1.2.4
miscellaneous | 1.3.1 | ...\\mingw64\\share\\octave\\packages\\miscellaneous-1.3.1
```

7.2 Файл main.m

```
main.m
1 % coding/main.m
2 % Подключение пакета signal:
3 pkg load signal;
4 % Входная кодовая последовательность:
5 data=[0 1 0 0 1 1 0 0 0 1 1 0];
6 % Входная кодовая последовательность для проверки свойства самосинхронизации
7 data_sync=[0 0 0 0 0 0 0 0 1 1 1 1 1 1];
8 % Входная кодовая последовательность для построения спектра сигнала:
9 data_spectre=[0 1 0 1 0 1 0 1 0 1 0 1 0 1];
10 % Создание каталогов signal, sync и spectre для размещения графиков:
11 mkdir 'signal';
12 mkdir 'sync';
13 mkdir 'spectre';
14 axis("auto");
```

Рисунок 22: Задаем входные кодовые последовательности

7.3 Файл main.m

```
15 % Униполярное кодирование
16 wave=unipolar(data);
17 plot(wave);
18 ylim([-1 6]);
19 title('Unipolar');
20 print 'signal/unipolar.png';
21 % Кодирование aml
22 wave=aml(data);
23 plot(wave);
24 title('AMI');
25 print 'signal/ami.png';
26 % Кодирование NRZ
27 wave=bipolarnrz(data);
28 plot(wave);
29 title('Bipolar Non-Return to Zero');
30 print 'signal/bipolarnrz.png';
31 % Кодирование RZ
32 wave=bipolarrz(data);
33 plot(wave);
34 title('Bipolar Return to Zero');
35 print 'signal/bipolarrz.png';
36 % Манчестерское кодирование
37 wave=manchester(data);
38 plot(wave);
39 title('Manchester');
40 print 'signal/manchester.png';
41
42 % Дифференциальное манчестерское кодирование
43 wave=diffmanc(data);
44 plot(wave);
45 title('Differential Manchester');
46 print 'signal/diffmanc.png';
```

Рисунок 23: Вызовы функций для построения модуляций кодированных сигналов кодовой последовательности data

7.4 Файл main.m

```
main.m
47 % Униполярное кодирование
48 wave=unipolar(data_sync);
49 plot(wave);
50 ylim([-1 6]);
51 title('Unipolar');
52 print 'sync/unipolar.png';
53 % Кодирование AMI
54 wave=ami(data_sync);
55 plot(wave);
56 title('AMI');
57 print 'sync/ami.png';
58 % Кодирование NRZ
59 wave=bipolarnrz(data_sync);
60 plot(wave);
61 title('Bipolar Non-Return to Zero');
62 print 'sync/bipolarnrz.png';
63 % Кодирование RZ
64 wave=bipolarrz(data_sync);
65 plot(wave);
66 title('Bipolar Return to Zero');
67 print 'sync/bipolarrz.png';
68 % Манчестерское кодирование
69 wave=manchester(data_sync);
70 plot(wave);
71 title('Manchester');
72 print 'sync/manchester.png';
73 % Дифференциальное манчестерское кодирование
74 wave=diffmanc(data_sync);
75 plot(wave);
76 title('Differential Manchester');
77 print 'sync/diffmanc.png';
```

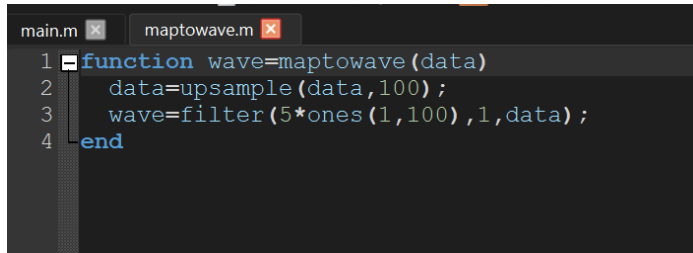
Рисунок 24: Вызовы функций для построения модуляций кодированных сигналов кодовой последовательности data_sync

7.5 Файл main.m

```
78 % Униполярное кодирование:
79 wave=unipolar(data_spectre);
80 spectre=calcspectre(wave);
81 title('Unipolar');
82 print 'spectre/unipolar.png';
83 % Кодирование AMI:
84 wave=ami(data_spectre);
85 spectre=calcspectre(wave);
86 title('AMI');
87 print 'spectre/ami.png';
88 % Кодирование NRZ:
89 wave=bipolarnrz(data_spectre);
90 spectre=calcspectre(wave);
91 title('Bipolar Non-Return to Zero');
92 print 'spectre/bipolarnrz.png';
93 % Кодирование RZ:
94 wave=bipolarrz(data_spectre);
95 spectre=calcspectre(wave);
96 title('Bipolar Return to Zero');
97 print 'spectre/bipolarrz.png';
98 % Манчестерское кодирование:
99 wave=manchester(data_spectre);
100 spectre=calcspectre(wave);
101 title('Manchester');
102 print 'spectre/manchester.png';
103 % Дифференциальное манчестерское кодирование:
104 wave=diffmanc(data_spectre);
105 spectre=calcspectre(wave);
106 title('Differential Manchester');
107 print 'spectre/diffmanc.png';
108
```

Рисунок 25: Вызовы функций для построения графиков спектров

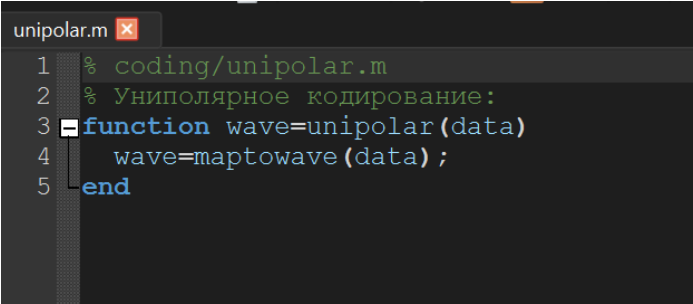
7.6 Файл mptowave.m



```
1 function wave=mptowave(data)
2     data=upsample(data,100);
3     wave=filter(5*ones(1,100),1,data);
4 end
```

Рисунок 26: Листинг файла mptowave.m

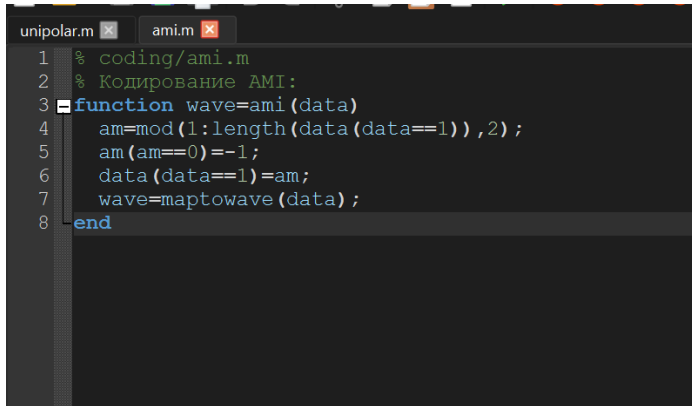
7.7 Файл unipolar.m



```
unipolar.m
1 % coding/unipolar.m
2 % Униполярное кодирование:
3 function wave=unipolar(data)
4     wave=maptowave(data);
5 end
```

Рисунок 27: Листинг файла unipolar.m

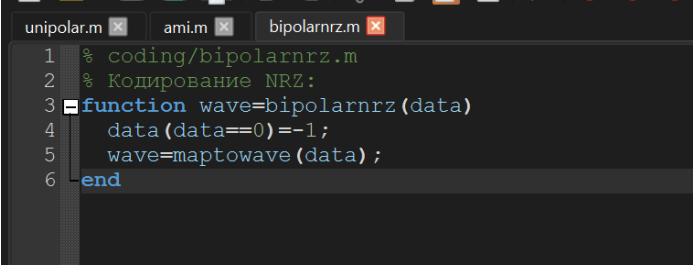
7.8 Файл ami.m



```
1 % coding/ami.m
2 % Кодирование AMI:
3 function wave=ami(data)
4     am=mod(1:length(data(data==1)),2);
5     am(am==0)=-1;
6     data(data==1)=am;
7     wave=maptowave(data);
8 end
```

Рисунок 28: Листинг файла ami.m

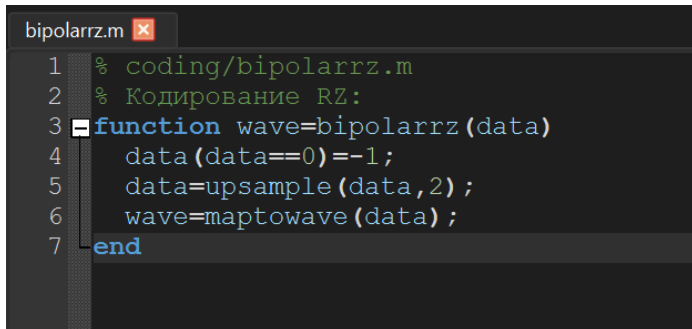
7.9 Файл bipolarnrz.m



```
1 % coding/bipolarnrz.m
2 % Кодирование NRZ:
3 function wave=bipolarnrz(data)
4     data(data==0)=-1;
5     wave=maptowave(data);
6 end
```

Рисунок 29: Листинг файла bipolarnrz.m

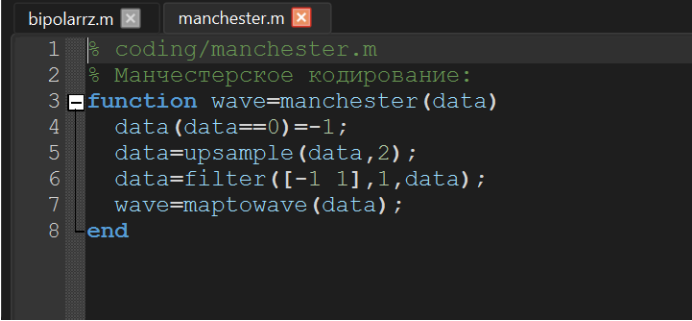
7.10 Файл bipolarrz.m



```
bipolarrz.m
1 % coding/bipolarrz.m
2 % Кодирование RZ:
3 function wave=bipolarrz(data)
4     data(data==0)=-1;
5     data=upsample(data,2);
6     wave=maptowave(data);
7 end
```

Рисунок 30: Листинг файла bipolarrz.m

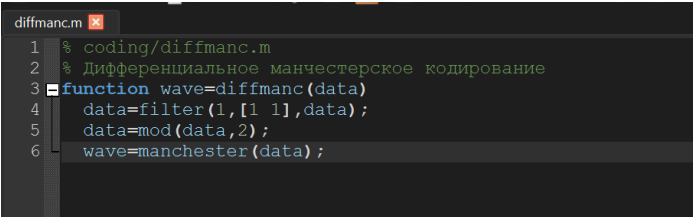
7.11 Файл manchester.m



```
bipolarrz.m x manchester.m x
1 % coding/manchester.m
2 % Манчестерское кодирование:
3 function wave=manchester(data)
4     data(data==0)=-1;
5     data=upsample(data,2);
6     data=filter([-1 1],1,data);
7     wave=maptowave(data);
8 end
```

Рисунок 31: Листинг файла manchester.m

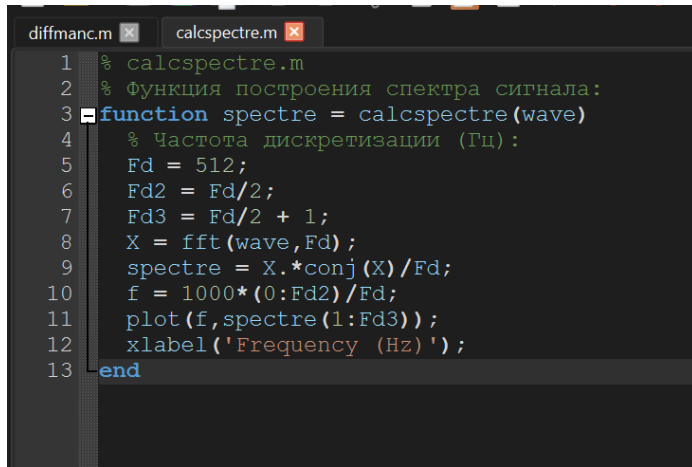
7.12 Файл diffmanc.m



```
diffmanc.m x
1 % coding/diffmanc.m
2 % Дифференциальное манчестерское кодирование
3 function wave=diffmanc(data)
4     data=filter(1,[1 1],data);
5     data=mod(data,2);
6     wave=manchester(data);
```

Рисунок 32: Листинг файла diffmanc.m

7.13 Файл calcspectre.m



```
1 % calcspectre.m
2 % Функция построения спектра сигнала:
3 function spectre = calcspectre(wave)
4 % Частота дискретизации (Гц):
5 Fd = 512;
6 Fd2 = Fd/2;
7 Fd3 = Fd/2 + 1;
8 X = fft(wave,Fd);
9 spectre = X.*conj(X)/Fd;
10 f = 1000*(0:Fd2)/Fd;
11 plot(f,spectre(1:Fd3));
12 xlabel('Frequency (Hz)');
13 end
```

Рисунок 33: Листинг файла calcspectre.m

7.14 График кодированного сигнала

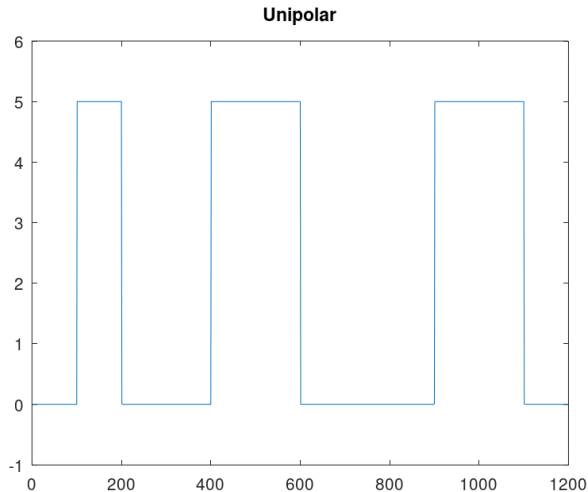


Рисунок 34: Униполярное кодирование

7.15 График кодированного сигнала

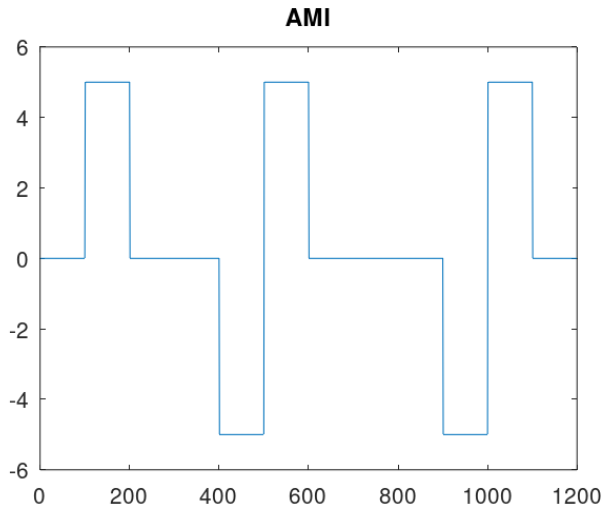


Рисунок 35: Кодирование AMI

7.16 График кодированного сигнала

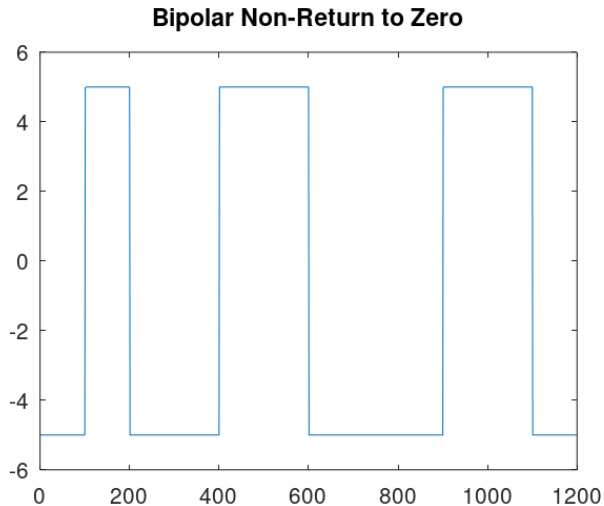


Рисунок 36: Кодирование NRZ

7.17 График кодированного сигнала

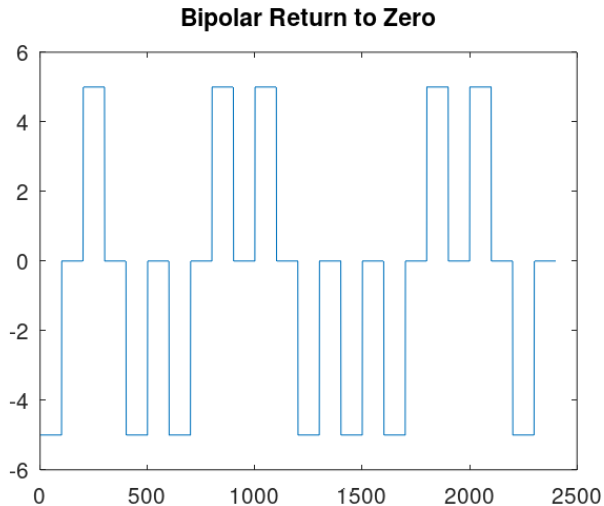


Рисунок 37: Кодирование RZ

7.18 График кодированного сигнала

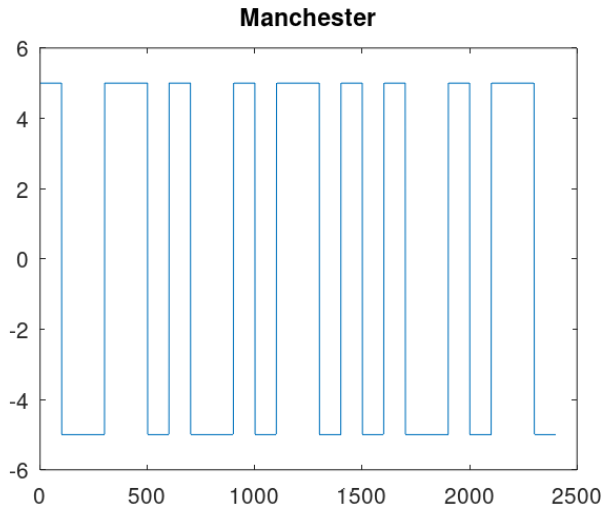


Рисунок 38: Манчестерское кодирование

7.19 График кодированного сигнала

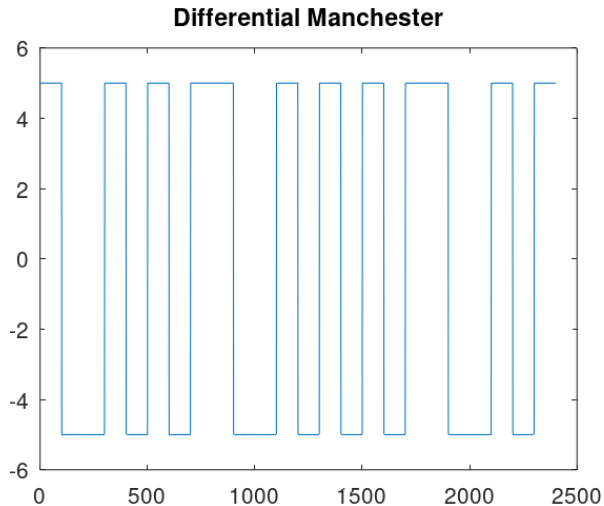


Рисунок 39: Дифференциальное манчестерское кодирование

7.20 Иллюстрация свойства самосинхронизации

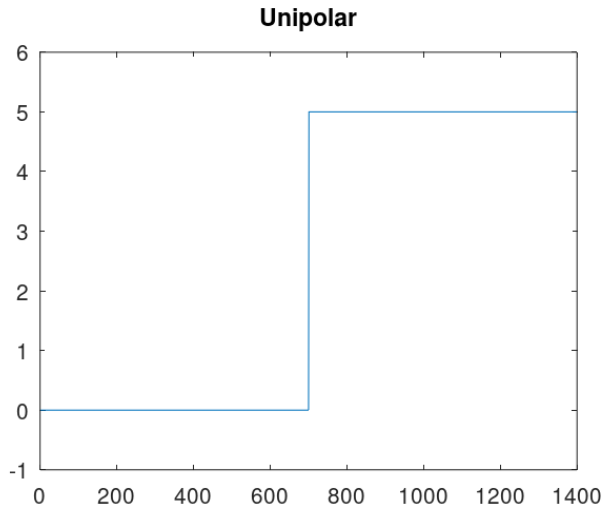


Рисунок 40: Униполярное кодирование: нет самосинхронизации

7.21 Иллюстрация свойства самосинхронизации

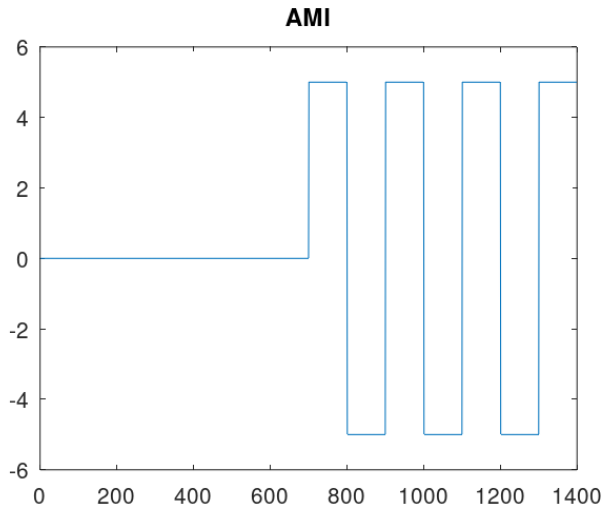


Рисунок 41: Кодирование AMI: самосинхронизация при наличии сигнала

7.22 Иллюстрация свойства самосинхронизации

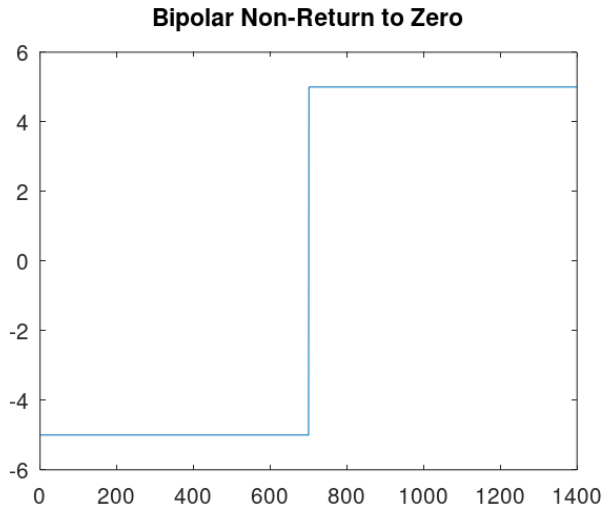


Рисунок 42: Кодирование NRZ: нет самосинхронизации

7.23 Иллюстрация свойства самосинхронизации

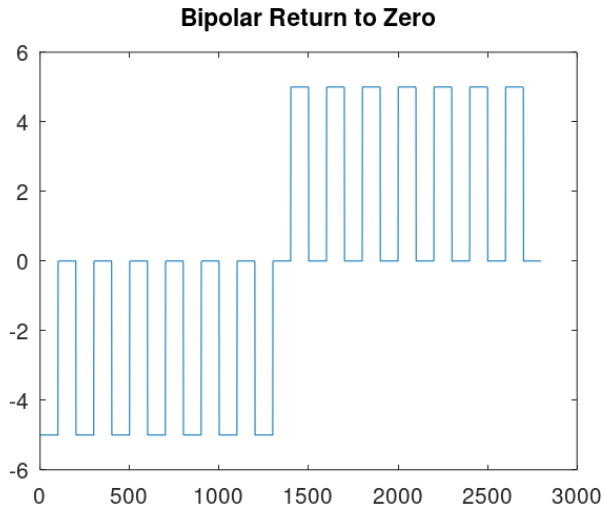


Рисунок 43: Кодирование RZ: есть самосинхронизация

7.24 Иллюстрация свойства самосинхронизации

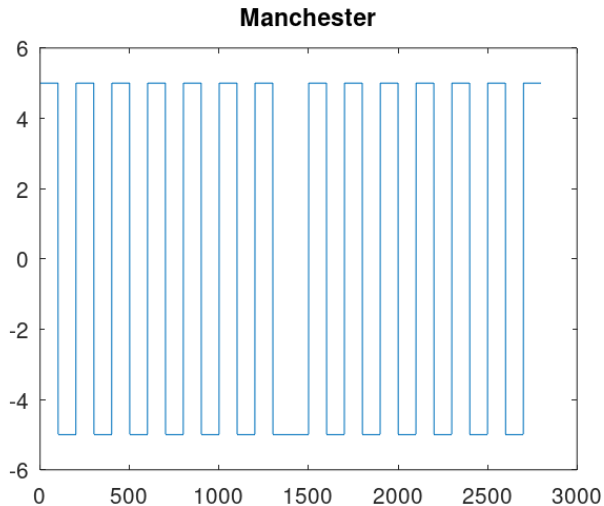


Рисунок 44: Манчестерское кодирование: есть самосинхронизация

7.25 Иллюстрация свойства самосинхронизации

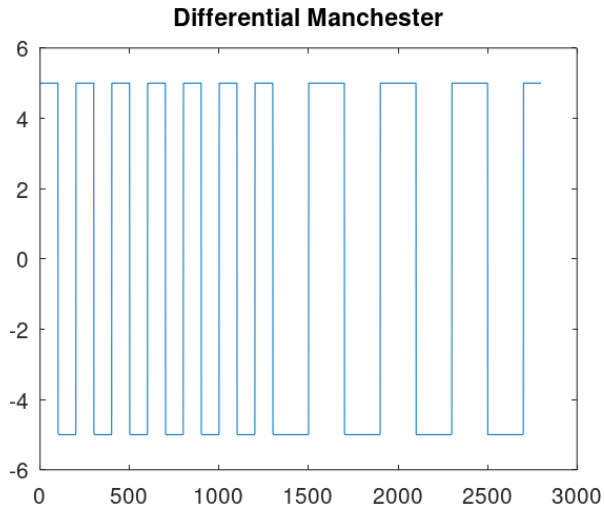


Рисунок 45: Дифференциальное манчестерское кодирование: есть самосинхронизация

7.26 Графики спектра сигнала

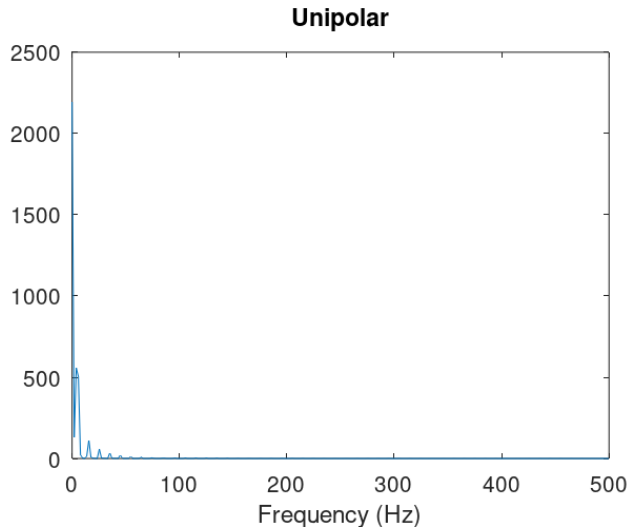


Рисунок 46: Униполярное кодирование: спектр сигнала

7.27 Графики спектра сигнала

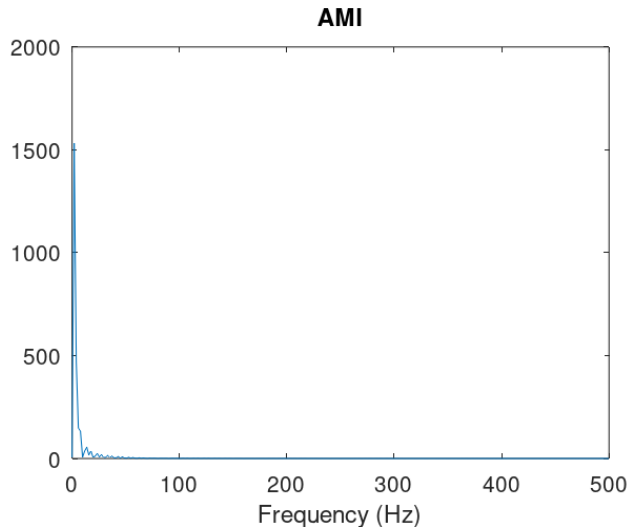


Рисунок 47: Кодирование AMI: спектр сигнала

7.28 Графики спектра сигнала

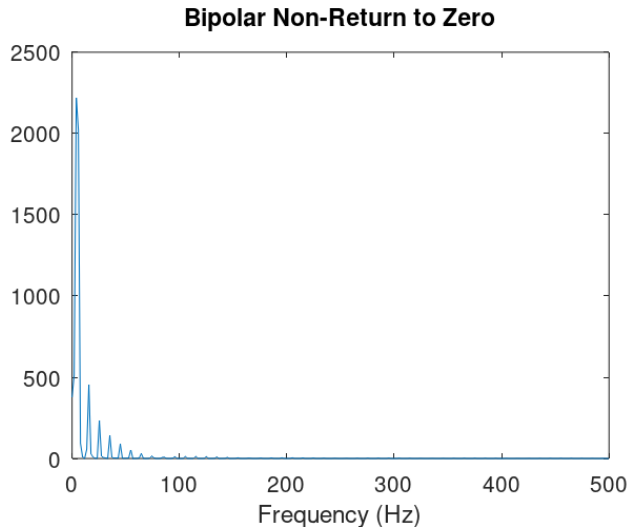


Рисунок 48: Кодирование NRZ: спектр сигнала

7.29 Графики спектра сигнала

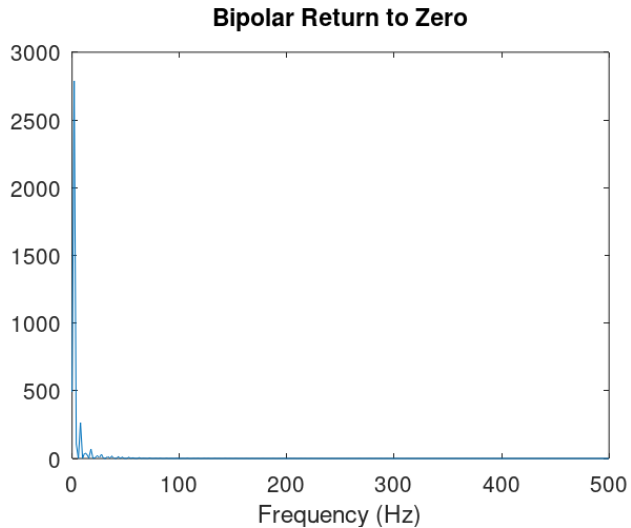


Рисунок 49: Кодирование RZ: спектр сигнала

7.30 Графики спектра сигнала

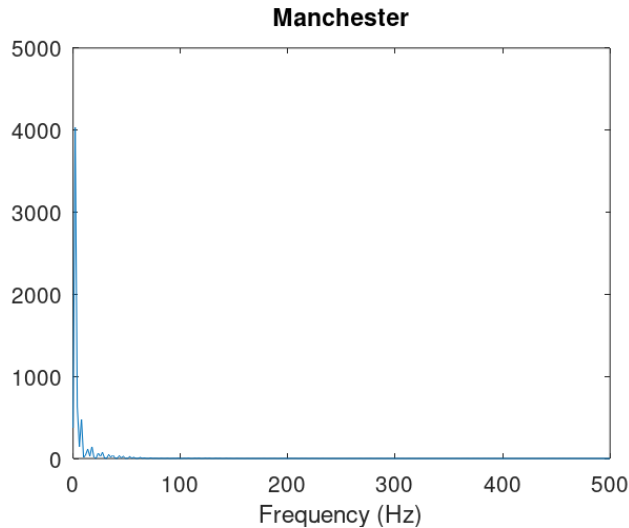


Рисунок 50: Манчестерское кодирование: спектр сигнала

7.31 Графики спектра сигнала

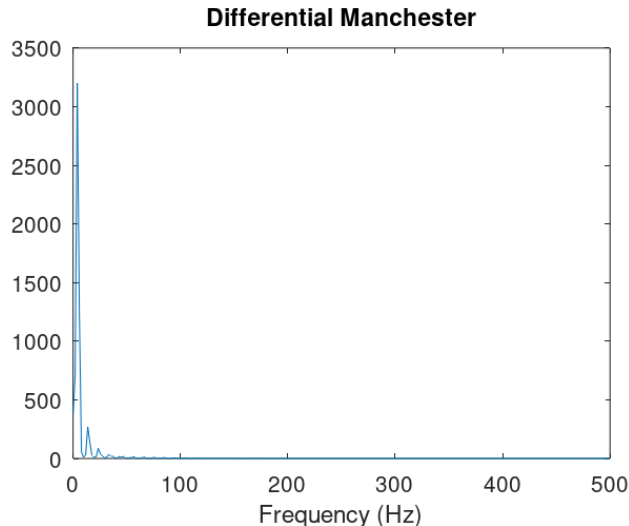


Рисунок 51: Дифференциальное манчестерское кодирование: спектр сигнала

- В ходе выполнения данной лабораторной работы я изучила методы кодирования и модуляции сигналов с помощью высокоуровневого языка программирования Octave. Определила спектр и параметры сигнала. Продемонстрировала принципы модуляции сигнала на примере аналоговой амплитудной модуляции. Исследовала свойства самосинхронизации сигнала.